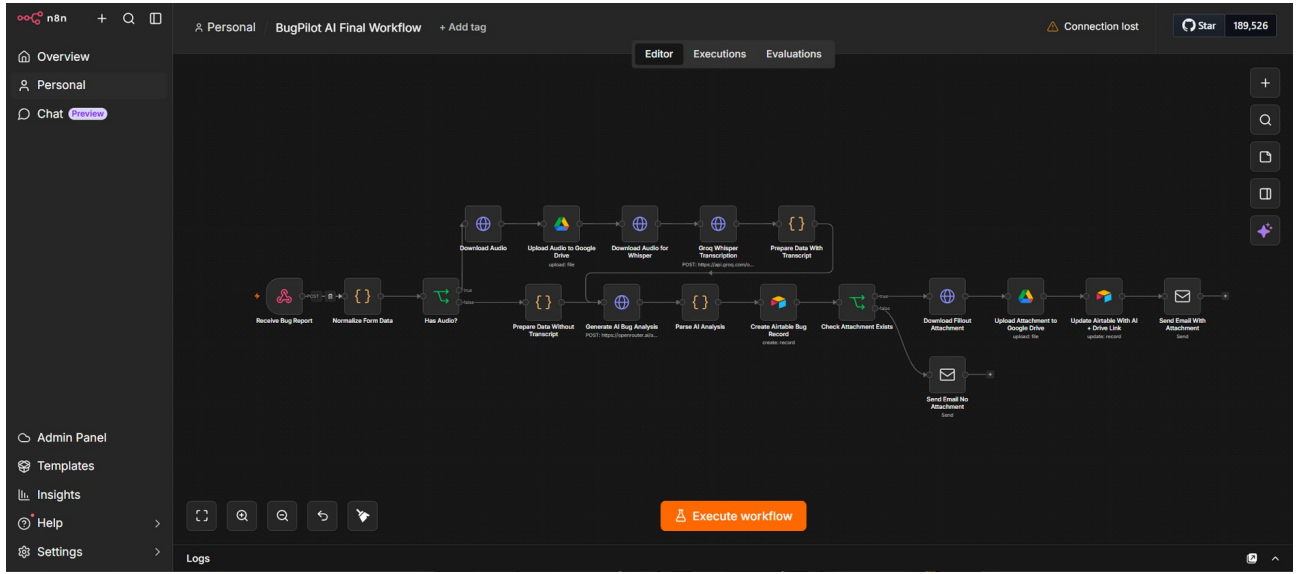


Bug Report → Solution Proposal AI

Başak Balkan
Student No: 244410022
Computer Engineering
Kastamonu University

Abstract- This study examines the BugPilot AI system developed to collect, structure, prioritize and forward user-reported bug records to the relevant channels within a single workflow. In the developed system, text, screenshots/files and optional voice recordings received through a Fillout form are processed in n8n; audio data is transcribed with the Groq Whisper service; bug content is analyzed with AI support through OpenRouter; results are recorded in Airtable; attachments are transferred to Google Drive; and a notification is sent via Gmail/SMTP at the end of the process. The application was validated through use cases with and without audio and attachments; the basic registration, transcription, classification, file storage and notification steps were observed to work consistently. The findings show that integrated automation architectures built with low-code tools can support bug reporting and initial triage processes in small-scale software teams.

Keywords- bug reporting, bug triage, workflow automation, n8n, OpenRouter, Groq Whisper, Airtable, Google Drive, Gmail



General view of the n8n workflow developed for BugPilot AI.

I. INTRODUCTION

In software development processes, bug reports are often received from different sources and in different formats. User e-mails, messaging applications, screenshots, short descriptions and audio recordings can be given as examples of these reports. Manual review, classification, prioritization and transfer of these data to the developer can cause both time loss and process inconsistencies. Therefore, converting bug reports into a standard structure and automating the first evaluation stage is important for software maintenance processes.

Within the scope of this study, the developed BugPilot AI system receives bug reports through a central form and processes the report content with an automated workflow.

B. Audio Transcription

The voice description field is optional. The Has Audio? IF node evaluates whether the audio_url field is filled. If an audio file exists, the content is downloaded in binary form with HTTP Request and uploaded to a Google Drive folder. It is converted to text by the Groq Whisper Transcription node. The Prepare Data With Transcript node adds the obtained text to the transcript field and makes it ready for AI analysis.

II. BACKGROUND AND TOOLS USED

BugPilot AI is based on an integrated architecture in which low-code automation tools and API-based services are combined. The n8n Webhook node starts the workflow with event data coming from external systems and supports the distinction between test and production URLs. Fillout was used as the data entry layer because it enables form submissions to be transferred to third-party systems through a webhook. Groq's speech-to-text service was preferred to convert audio data into text. The OpenRouter chat completions interface was used in the stage of summarizing the bug report and producing structured output. Airtable personal access token infrastructure was used to store bug records in a table structure. The Google Drive API was used for storing files and generating shareable links, while Gmail/SMTP configuration was used for sending notifications. By using these components together, the manual bug reporting process was transformed into a triage pipeline whose initial evaluation is performed automatically. Bug triage and bug fix time prediction studies in the literature show that keeping bug records organized, structured and traceable supports later decision processes. In this direction, the developed system combines data collection, analysis, recording, file management and notification steps in a single operational flow.

III. SYSTEM ARCHITECTURE

The general architecture of the system was designed as an event-based workflow. When the Fillout form is submitted, the production webhook defined in n8n is triggered. In the first stage, the Normalize Form Data node transforms the question-answer structure sent by Fillout into fixed fields such as name, email, title, description, attachment_url, audio_url and transcript. This normalization layer prevents later steps from being directly dependent on the raw data structure of the form service. If an audio file is present, the flow is directed to the true branch: the audio file is downloaded, saved to the Google Drive folder, sent to the Groq Whisper service and the resulting transcript field is included in the AI analysis. When no audio recording is present, the transcript field is left empty and the process continues without interruption. Similarly, when a screenshot or attachment is present, the Google Drive upload step runs; when no file is present, the system moves to the notification branch without a file and completes without producing an error. The added visual shows the final state of this branching structure in the n8n interface.

IV. IMPLEMENTATION

A. Form Intake and Normalization

The Fillout form consists of user information, bug details, device/browser information, screenshot/file and voice description fields. The Normalize Form Data code node in n8n converts the nested question-answer structure sent by Fillout into standard JSON fields. Thanks to this layer, later node steps work independently of the raw structure of the form fields and in a more stable manner. It uses the description, reproduction steps, expected and actual result, device/browser information, optional file and optional voice description fields. The flow created in n8n transforms these data into standard JSON fields; when audio data is present, it produces a transcription, interprets the bug content with an AI model and transfers the results to the database, file storage and e-mail notification layers. The main objective of the study is not only to transfer data between services, but also to create a traceable and extensible bug reporting infrastructure that can operate without interruption under real usage conditions such as missing attachments, the absence of audio recordings or different input combinations.

C. AI-Supported Bug Analysis

Generate AI Bug Analysis node sends a structured JSON request to the OpenRouter chat completions endpoint. The prompt includes the reporter, product/page, bug title, description, reproduction steps, expected/actual result, device/browser information and, if available, the audio transcript. The model was asked to return directly valid JSON instead of free text. Parse AI Analysis node cleans this response and separates it into summary, category, severity, possible_cause, suggested_solution, developer_notes, missing_information and user_reply fields. Thus, a standard data structure is used in the Airtable and e-mail steps.

D. Data Storage and Notification

The Create Airtable Bug Record node writes the basic bug information, transcript field and text fields of the AI analysis outputs to the Airtable Bug Reports table. The Check Attachment Exists node checks the file field. If a file exists, the relevant content is downloaded, uploaded to the BugPilot AI Attachments folder in Google Drive, and a Drive link is added to the Attachment URL field in the Airtable record. If no file exists, the Attachment URL field remains empty. In both cases, the result notification is sent via Gmail/SMTP.

V. TEST SCENARIOS AND RESULTS

The system was manually tested through three basic use scenarios: (i) a bug report with file and audio, (ii) a bug report without a file but with audio, and (iii) a bug report without file or audio. In each scenario, the n8n execution result, creation of the Airtable record, Google Drive file outputs, production of the audio transcript and sending of the e-mail notification were checked separately.

The test results show that the system is not dependent on a single successful path. When the file or audio fields were left blank, the relevant IF branches were activated and the workflow was completed without interruption. This is important for real usage conditions in which end users do not fill in every field completely when submitting a report. In addition, it was observed that the transcript produced in audio scenarios enriched the bug description and provided additional context for the analysis step.

VI. DISCUSSION

BugPilot AI should not be evaluated as a monitoring system that performs fully automatic bug discovery, but as a system that transforms manual bug report input into an automated triage and distribution process. The system structures the problem reported by the user, makes it more readable and traceable for the developer, transfers the relevant attachments to central repositories and produces a notification. Bug triage studies in the literature also show that correctly structuring bug records supports decision-making and responsibility assignment processes.

Another important point architecturally is that API keys are not embedded in plain-text outputs or form data and are stored in n8n credential fields. In addition, the use of the production webhook URL eliminates dependency on the test URL and enables the system to work automatically in real form submissions. This preference is a critical implementation decision in terms of both continuity and security.

VII. LIMITATIONS AND FUTURE WORK

The outputs of the current system depend on the scope and clarity of the information provided in the bug report. Missing or ambiguous user explanations can be marked by the system in the missing_information field; however, the generated analysis does not prove the exact code-level cause of the bug by itself. In the future, the system can be extended with additions such as application logs, screen recording analysis, automatic issue creation, Slack/Discord notifications and support conversation routing.

In future work, it is also important to evaluate system outputs with quantitative quality metrics. For example, comparing severity predictions with human assessments, measuring category consistency and checking false positive/negative analyses, if any, can increase system reliability. In addition, as long-term usage data accumulates, additional decision support modules such as bug triage and fix time prediction can be developed.

VIII. CONCLUSION

In this study, the BugPilot AI system, which combines the tasks of collecting, structuring, analyzing and delivering bug reports to the relevant people in a single workflow by using Fillout, n8n, OpenRouter, Groq Whisper, Airtable, Google Drive and Gmail/SMTP, was designed and implemented. The developed solution performs the steps of normalizing form data, generating audio transcripts, obtaining structured analysis output, creating Airtable records, storing files in Google Drive and sending e-mail notifications end to end. The successful results obtained in test scenarios with and without attachments and with and without audio show that the system offers an infrastructure that is robust against real user behavior and can be improved.

REFERENCES

- [1] n8n Docs, "Webhook node," n8n Documentation. [Online]. Available: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>
- [2] Fillout Help, "Send form submissions to a webhook," Fillout Documentation. [Online]. Available: <https://fillout.com/help/webhook>
- [3] Groq, "Speech to Text," GroqDocs. [Online]. Available: <https://console.groq.com/docs/speech-to-text>
- [4] OpenRouter, "Create a chat completion," OpenRouter Documentation. [Online]. Available: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
- [5] Airtable Support, "Creating personal access tokens," Airtable Documentation. [Online]. Available: <https://support.airtable.com/docs/creating-personal-access-tokens>
- [6] Google, "Google Drive API overview," Google for Developers. [Online]. Available: <https://developers.google.com/workspace/drive/api/guides/about-sdk>
- [7] Google, "Add Gmail to another email client," Gmail Help. [Online]. Available: <https://support.google.com/mail/answer/7126229>
- [8] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in Proceedings of the 28th International Conference on Software Engineering (ICSE'06), Shanghai, China, 2006, pp. 361-370, doi: 10.1145/1134285.1134336.
- [9] E. Giger, M. D'Ambros, M. Pinzger, and H. C. Gall, "Predicting the Fix Time of Bugs," in Proceedings of the 2nd International Workshop on Recommendation Systems for Software Engineering, Cape Town, South Africa, 2010, pp. 52-56, doi: 10.1145/1808920.1808933.
- [10] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), Atlanta, GA, USA, 2008, pp. 308-318, doi: 10.1145/1453101.1453146.
- [11] S. Breu, R. Premraj, J. Sillito, and T. Zimmermann, "Information needs in bug reports: improving cooperation between developers and users," in Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW), Savannah, GA, USA, 2010, pp. 301-310, doi: 10.1145/1718918.1718973.
- [12] P. Runeson, M. Alexandersson, and O. Nyholm, "Detection of duplicate defect reports using natural language processing," in Proceedings of the 29th International Conference on Software Engineering (ICSE), Minneapolis, MN, USA, 2007, pp. 499-510, doi: 10.1109/ICSE.2007.32.
- [13] N. Jalbert and W. Weimer, "Automated duplicate detection for bug tracking systems," in Proceedings of the IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN), Anchorage, AK, USA, 2008, pp. 52-61, doi: 10.1109/DSN.2008.4630070.
- [14] C. Sun, D. Lo, S.-C. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering (ASE), Lawrence, KS, USA, 2011, pp. 253-262.
- [15] J. Zhou, H. Zhang, and D. Lo, "Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports," in Proceedings of the 34th International Conference on Software Engineering (ICSE), Zurich, Switzerland, 2012, pp. 14-24, doi: 10.1109/ICSE.2012.6227210.
- [16] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry, "Improving bug localization using structured information retrieval," in Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), Palo Alto, CA, USA, 2013, pp. 345-355.